

HW3

Diagram

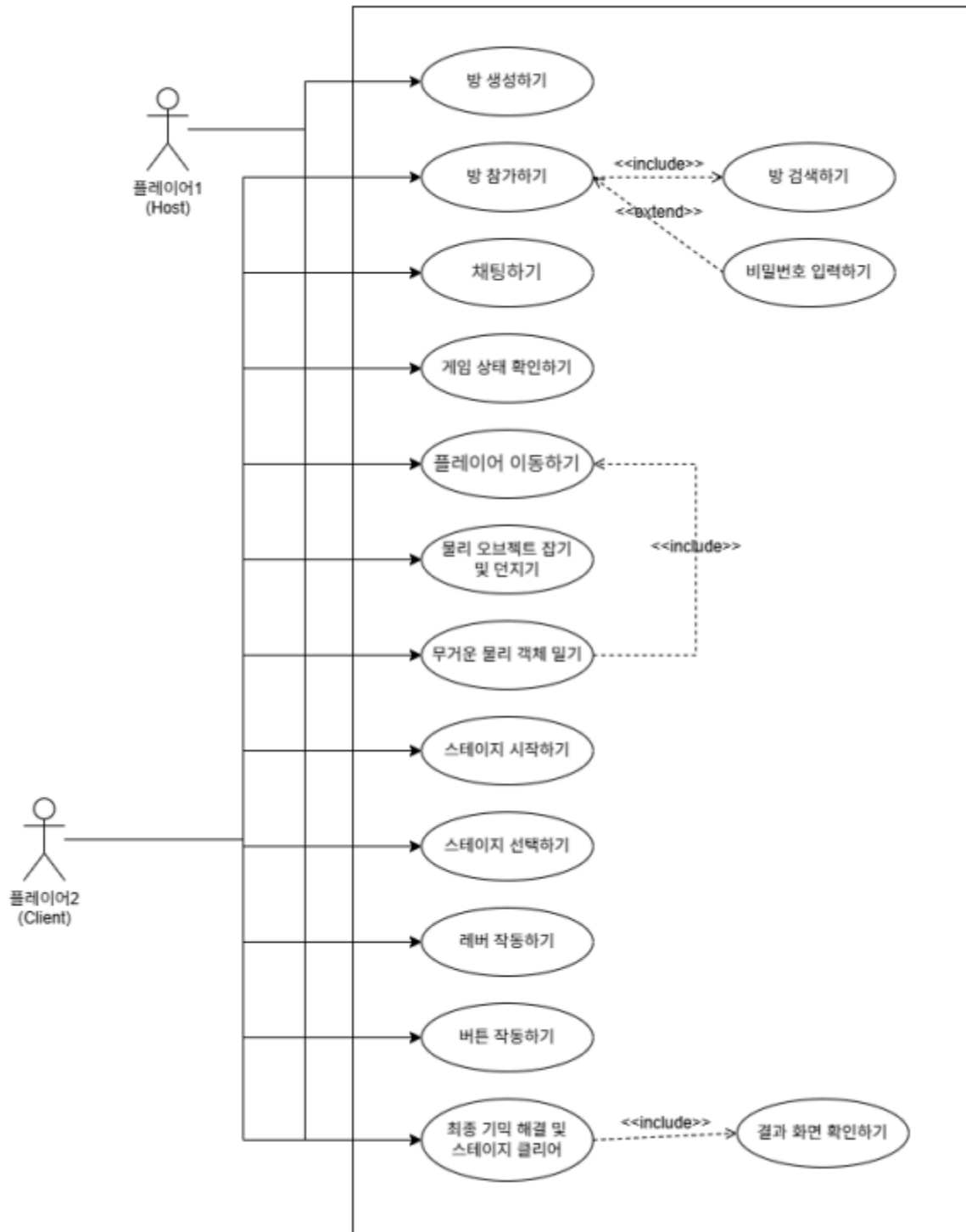
팀명: It maybe takes two

팀원:

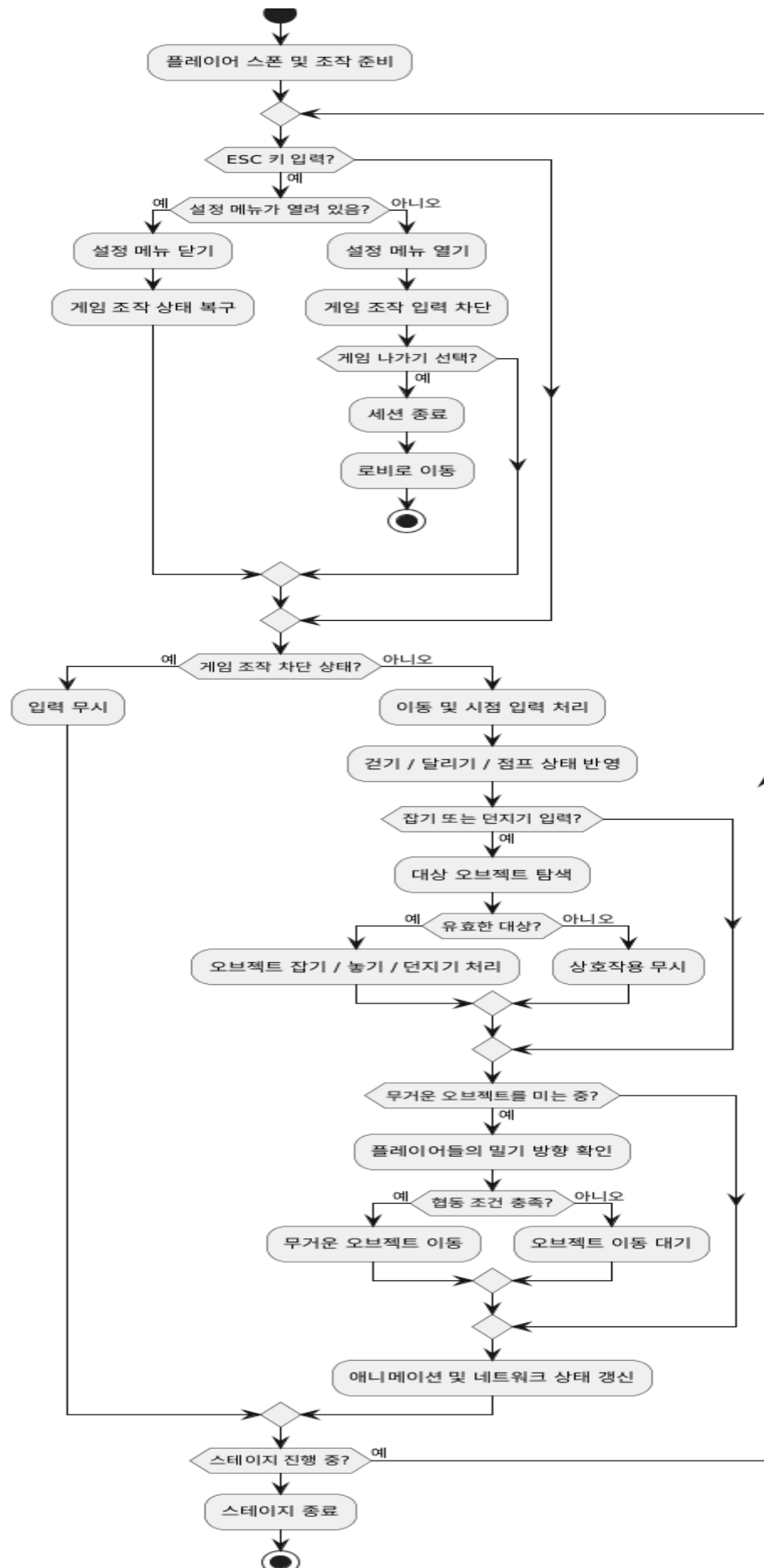
- 20223085 배상혁
- 20220611 이규원
- 20223104 양준영

3. Use Case Diagram

3. Use Case Diagram

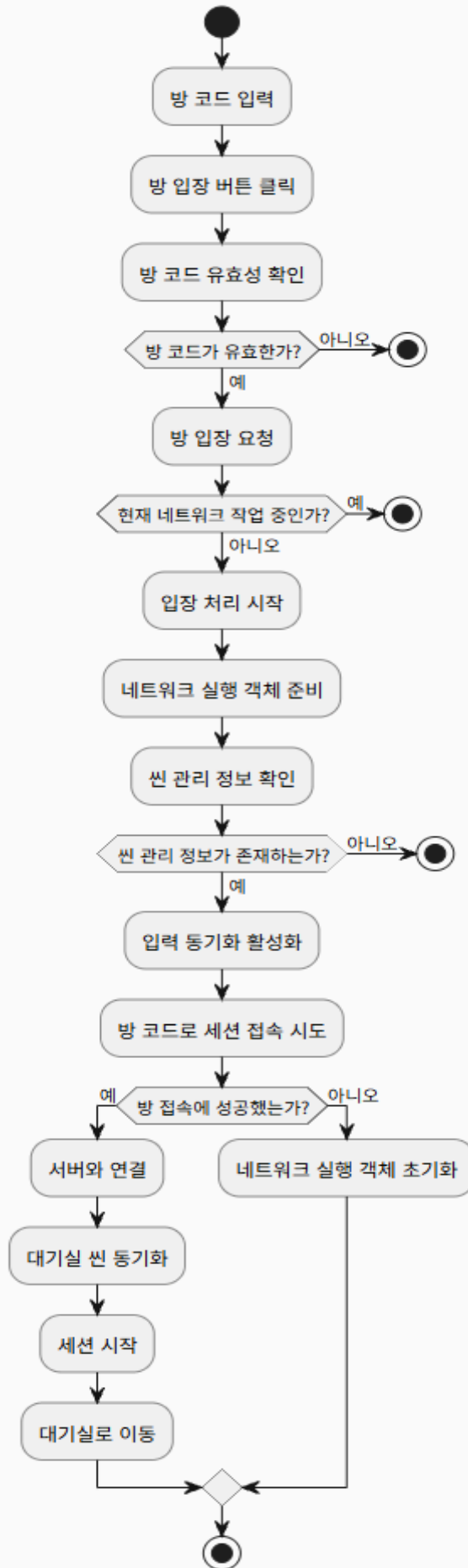


5. Activity Diagram Player

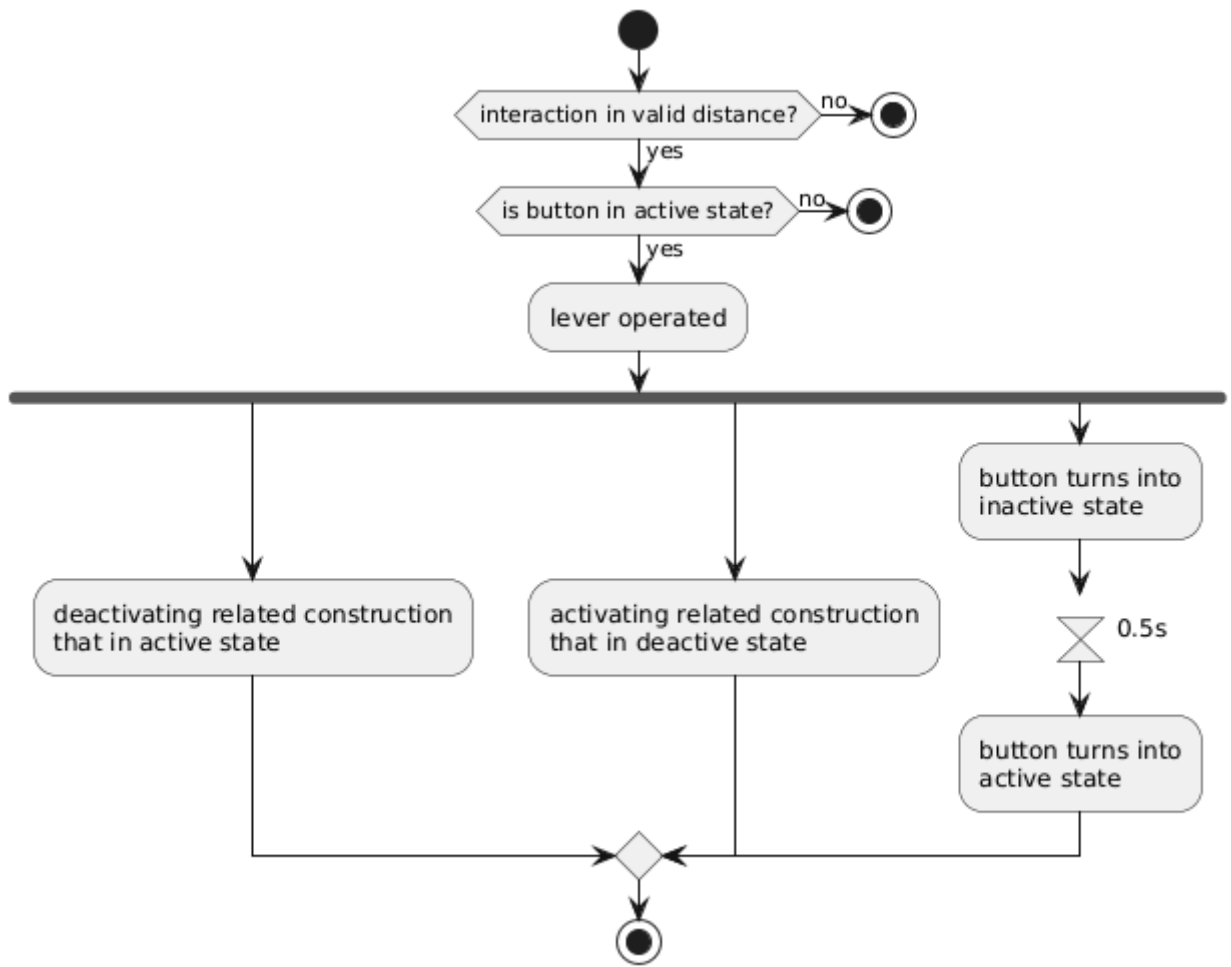


Server

방 입장(Room Join) 유즈케이스 Activity Diagram

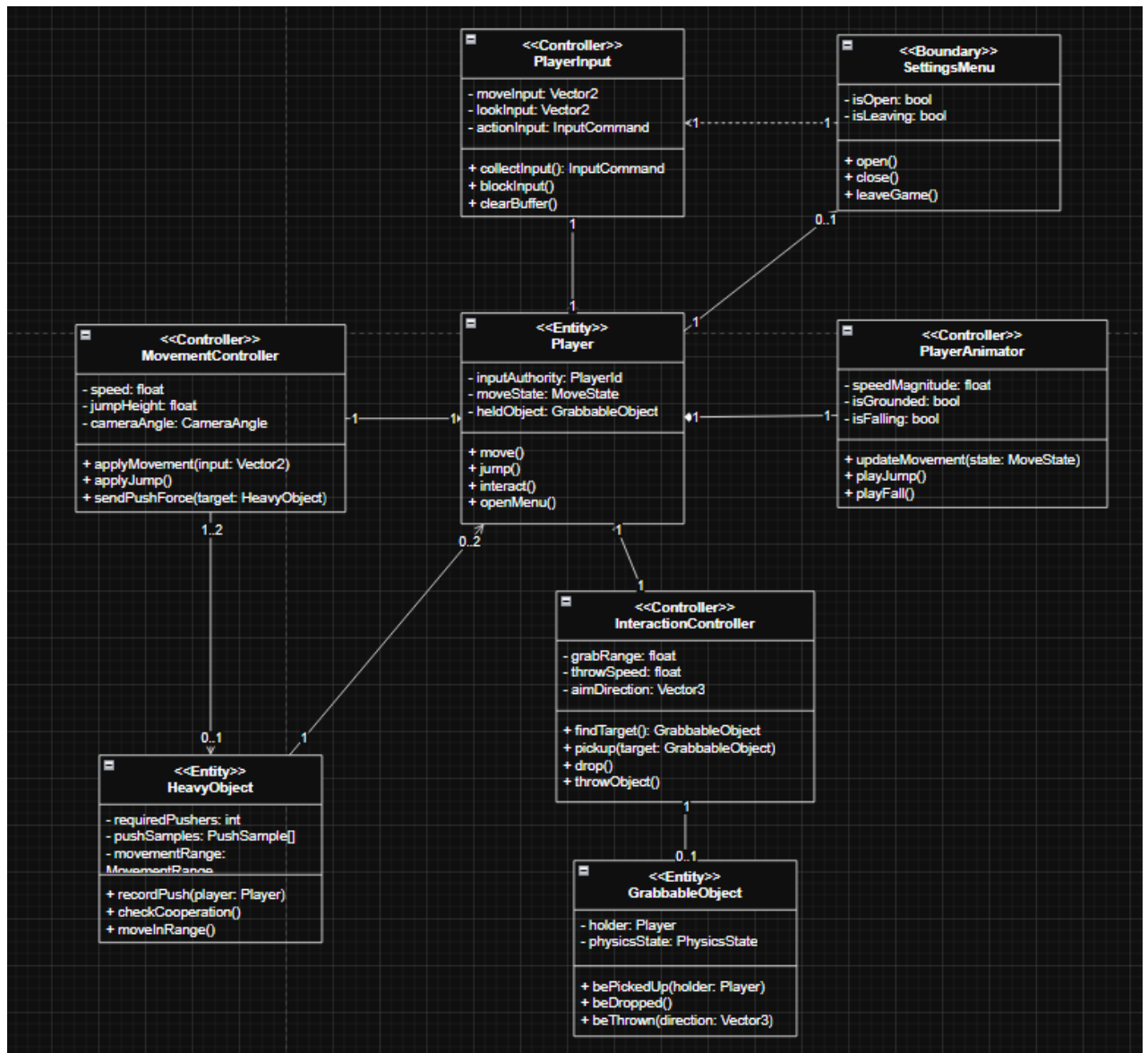


Object

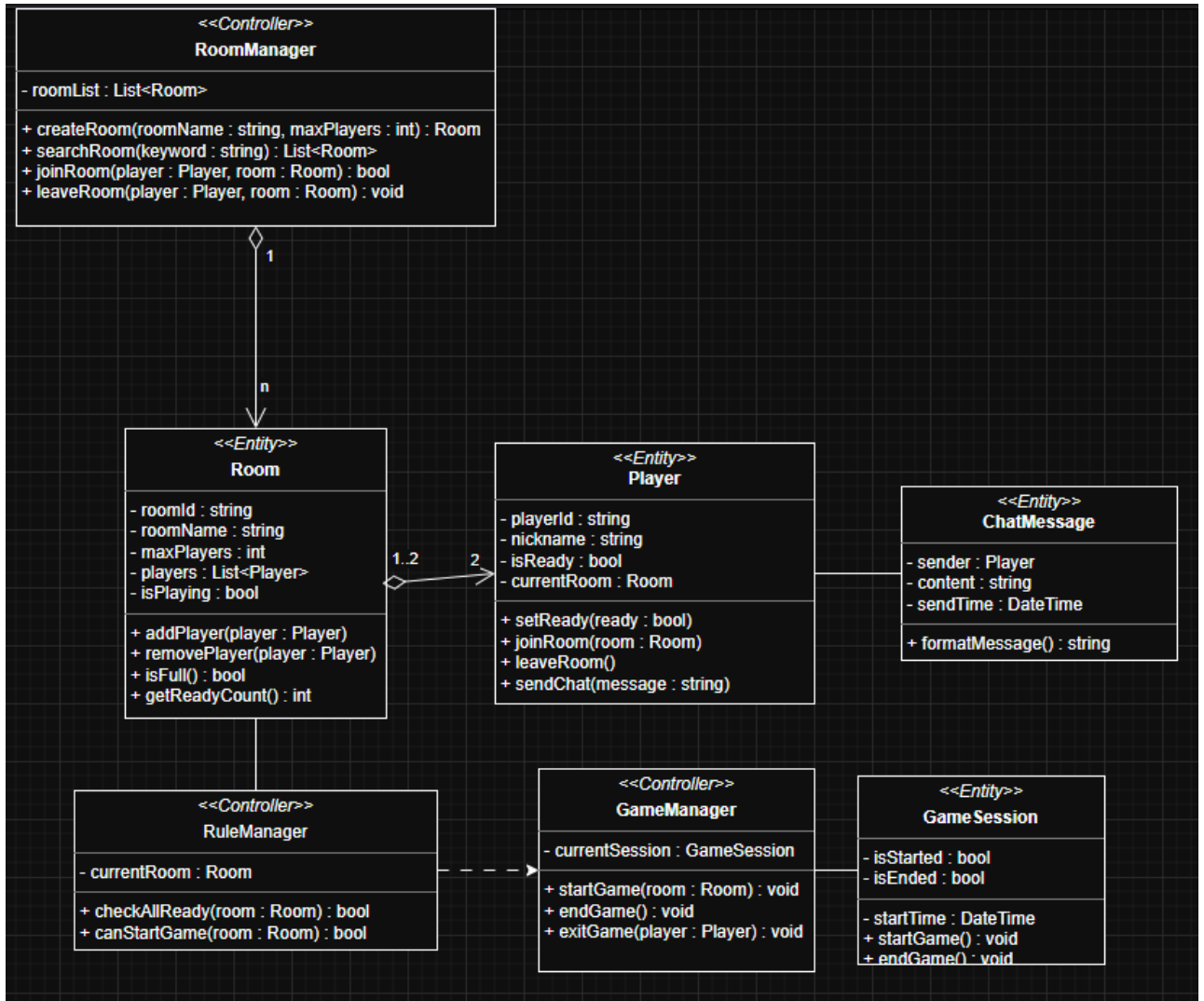


6. Conceptual Class Diagram

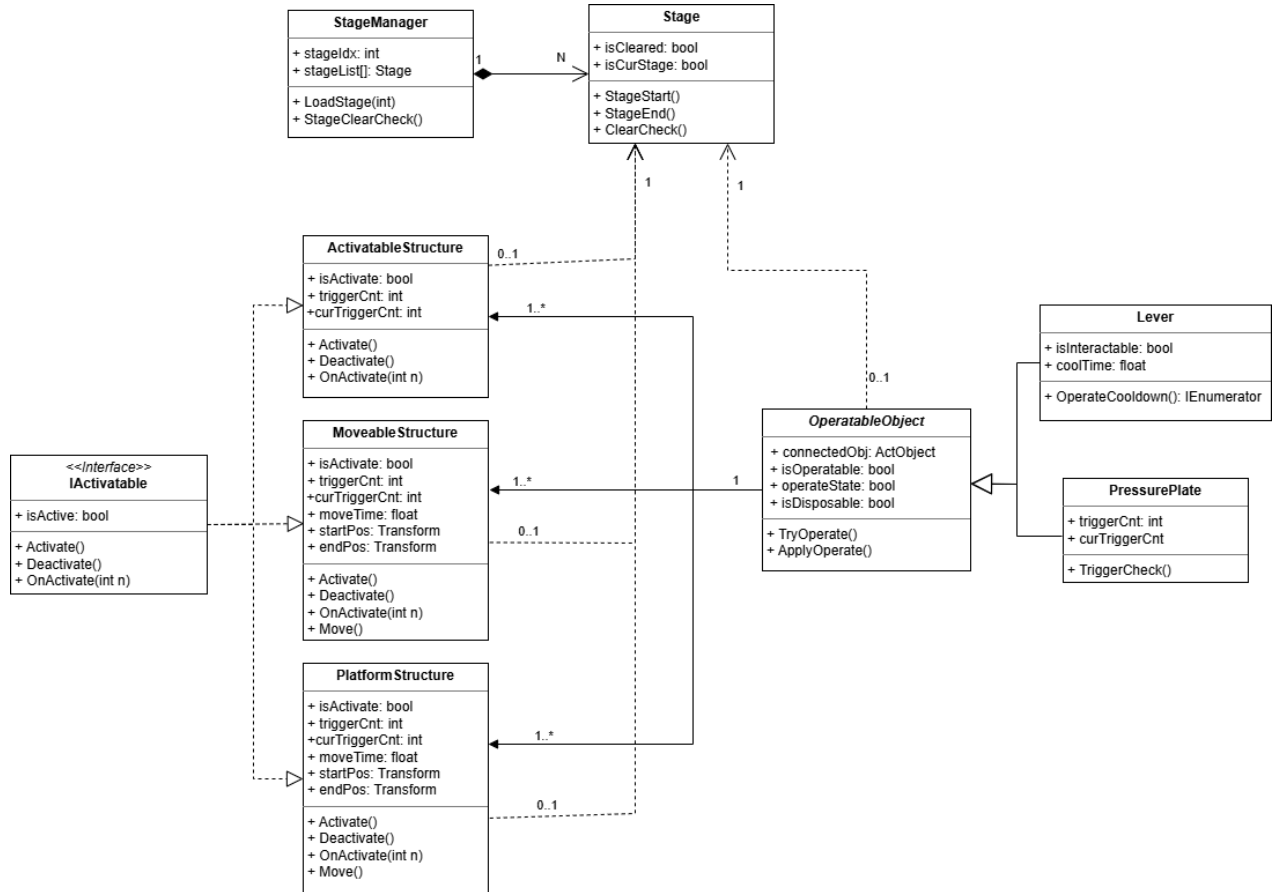
Player



Server



Object



7-1. System Sequence Diagram

시스템 시퀀스 다이어그램



7-1. Operation Contract

System Operation	Responsibility	Pre-condition	Post-condition
<code>createRoom(roomName, maxPlayers, isPublic)</code>	플레이어 1이 새 방 생성을 요청하고 대기실 입장을 시작한다.	플레이어 1이 로비에 있으며 방 생성에 필요한 값을 입력했다.	방 생성 성공 시 <code>enterWaitingRoom(roomId)</code> 가 반환된다. 실패 시 <code>showCreateRoomError(reason)</code> 가 반환된다.
<code>joinRoom(roomId)</code>	플레이어 2가 기존 방 참가를 요청하고 대기실 입장을 시작한다.	플레이어 2가 로비에 있으며 참가할 <code>roomId</code> 를 입력했다.	참가 성공 시 <code>showWaitingRoom(roomInfo)</code> 가 반환된다. 실패 시 <code>joinRoomRejected(reason)</code> 가 반환된다.
<code>sendChatMessage(message)</code>	대기실에서 플레이어 메시지를 같은 방 참가자에게 전달한다.	플레이어가 방에 입장이 있고 채팅 입력이 가능하다.	메시지가 플레이어 1, 플레이어 2의 채팅 화면에 표시된다. 처리할 수 없으면 요청자에게 거절 결과가 반환된다.
<code>setReady(playerId, isReady)</code>	각 플레이어의 준비 상태를 대기실 상태에 반영한다.	플레이어가 대기실에 입장이 있다.	해당 플레이어의 준비 상태가 갱신되고 대기실 참가자에게 공유된다.
<code>requestStartStage()</code>	플레이어 1이 스테이지 시작을 요청한다.	플레이어 1이 방장 역할이며 스테이지를 시작할 수 있는 대기실 상태이다.	시작 가능하면 두 플레이어에게 <code>loadStage(stageId)</code> 가 전달된다. 시작할 수 없으면 플레이어 1에게 <code>startStageRejected(reason)</code> 가 반환된다.
<code>enterStage(stageId)</code>	플레이어가 스테이지 입장을 요청하고 게임 진행 상태로 전환한다.	스테이지 시작 요청이 수락되었고 <code>stageId</code> 가 정해져 있다.	입장 성공 시 <code>stageReady(stageId)</code> 가 반환된다. 실패 시 <code>returnToLobbyWithError(reason)</code> 가 반환된다.

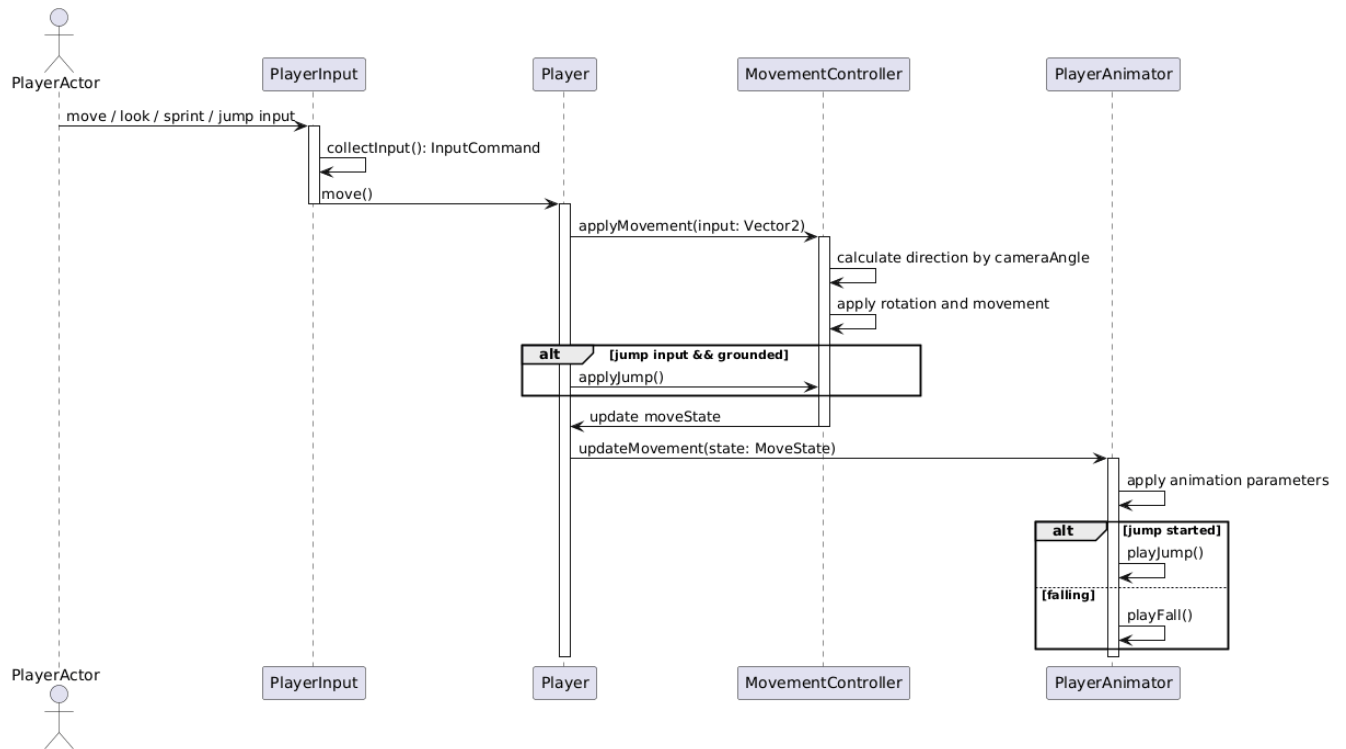
System Operation	Responsibility	Pre-condition	Post-condition
<code>inputMovement()</code>	플레이어 이동 입력을 스테이지 상태에 반영한다.	스테이지가 진행 중이고 플레이어 조작이 가능한 상태이다.	플레이어 위치와 화면에 반영될 상태가 <code>updateStageState(result)</code> 로 갱신된다.
<code>requestGrab()</code> <code>operateLever()</code> <code>toggleStageSettingsMenu()</code>	플레이어 1의 오브젝트 잡기, 레버 조작, 설정 메뉴 요청을 처리한다.	플레이어 1이 스테이지 안에 있으며 대상 오브젝트나 UI를 조작할 수 있다.	오브젝트, 구조물, 메뉴 상태가 변경되고 결과가 <code>updateStageState(result)</code> 로 반환된다.
<code>pushObject()</code> <code>operateButton()</code> <code>stepOnPressurePlate()</code>	플레이어 2의 협동 밀기, 버튼 조작, 발판 활성화 요청을 처리한다.	플레이어 2가 스테이지 안에 있으며 협동 기믹 수행 위치에 있다.	협동 오브젝트, 다리, 최종 발판 상태가 변경되고 결과가 <code>updateStageState(result)</code> 로 반환된다.
<code>enterExitArea()</code>	플레이어의 출구 진입을 기록하고 스테이지 클리어 결과를 반환한다.	출구가 열려 있으며 플레이어가 출구 영역에 진입했다.	두 플레이어가 모두 진입하면 <code>stageCleared()</code> 가 반환된다. 일부만 진입한 경우 <code>waitForPartner()</code> 가 반환된다.

7-2. Sequence Diagram

Player

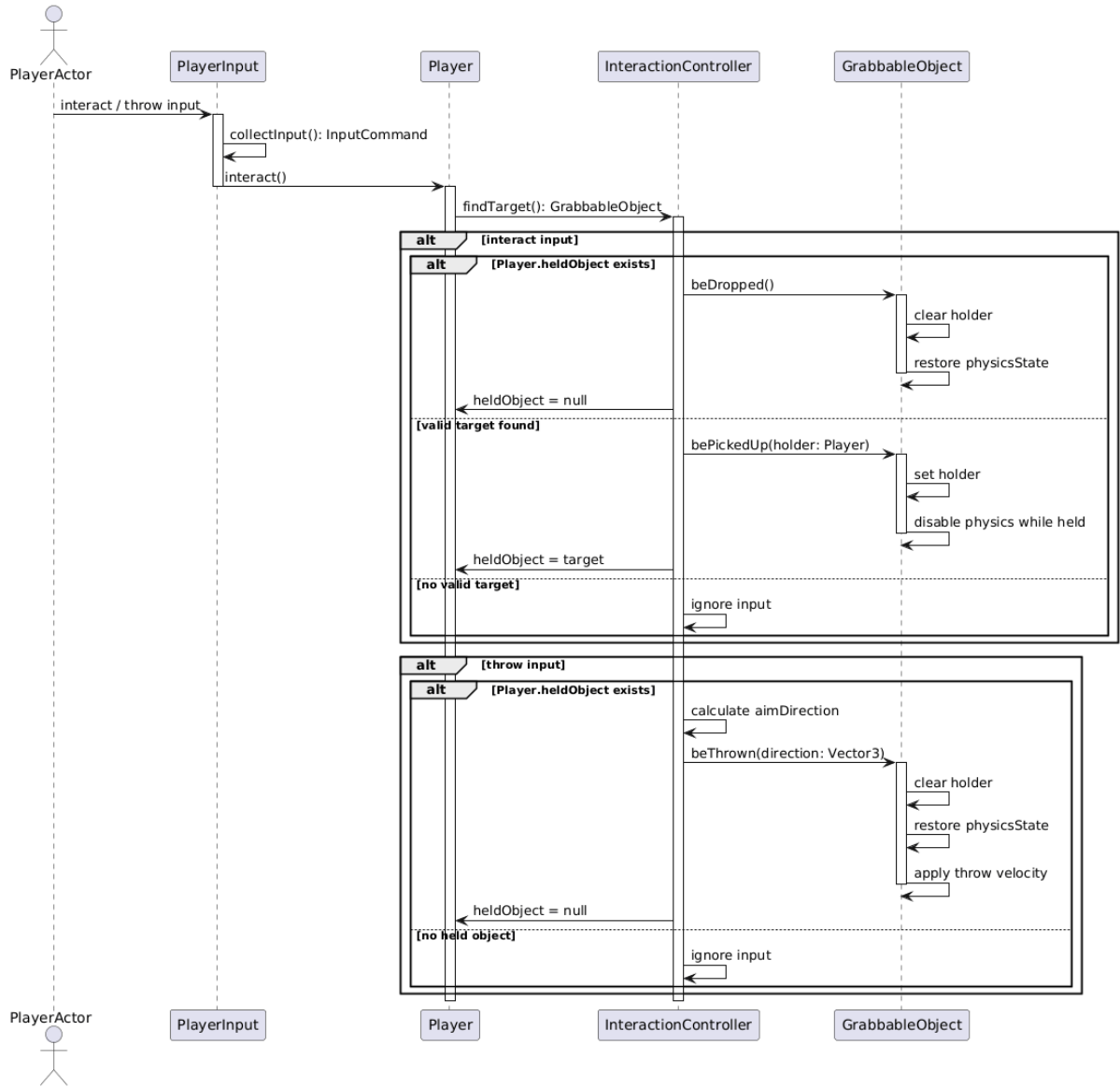
UseCase 1: 플레이어 이동 및 시점 조작

Use Case 1 - Player Movement and View Control



UseCase 2: 물리 오브젝트 잡기/놓기/던지기

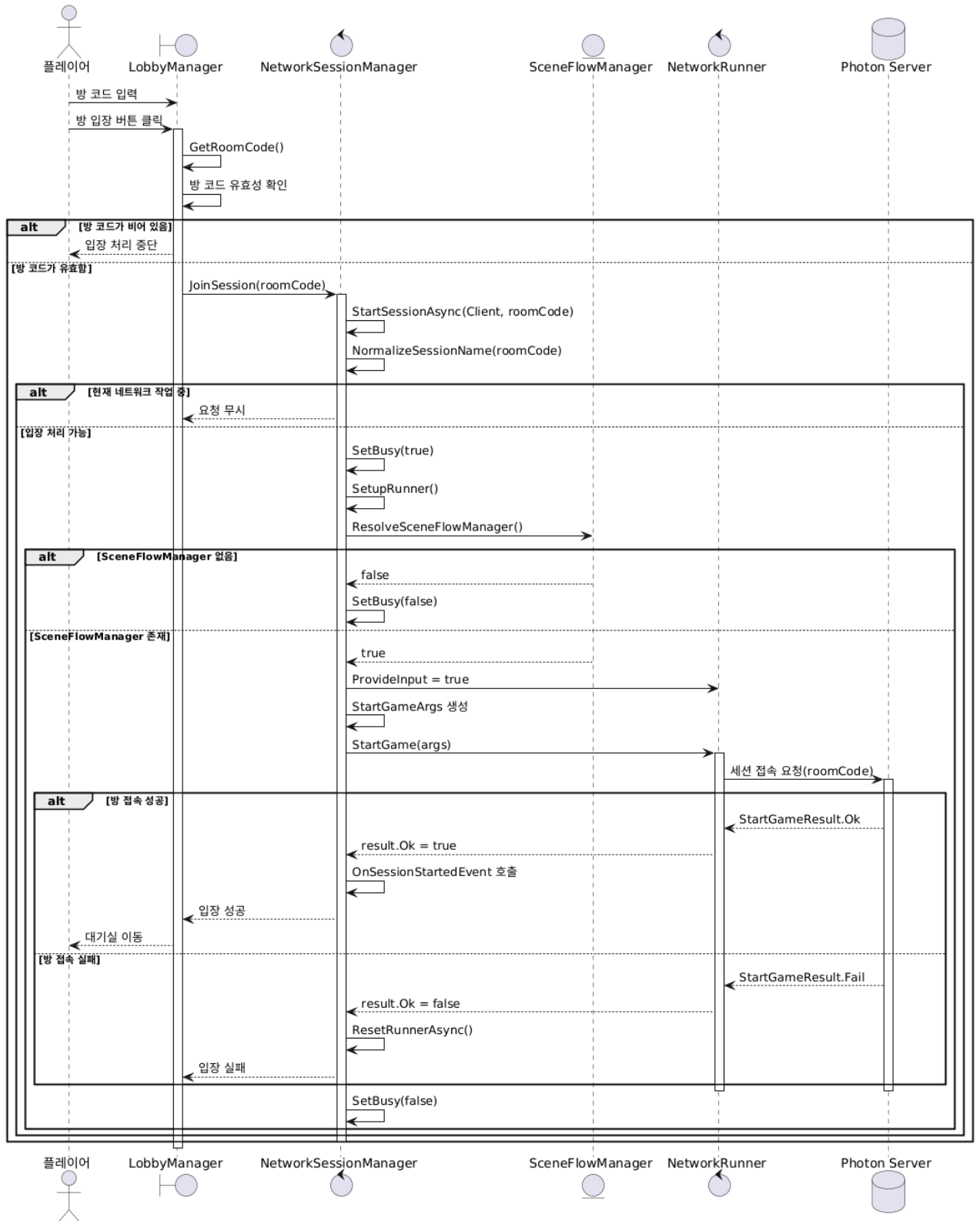
Use Case 2 - Grabbable Object Pickup / Drop / Throw



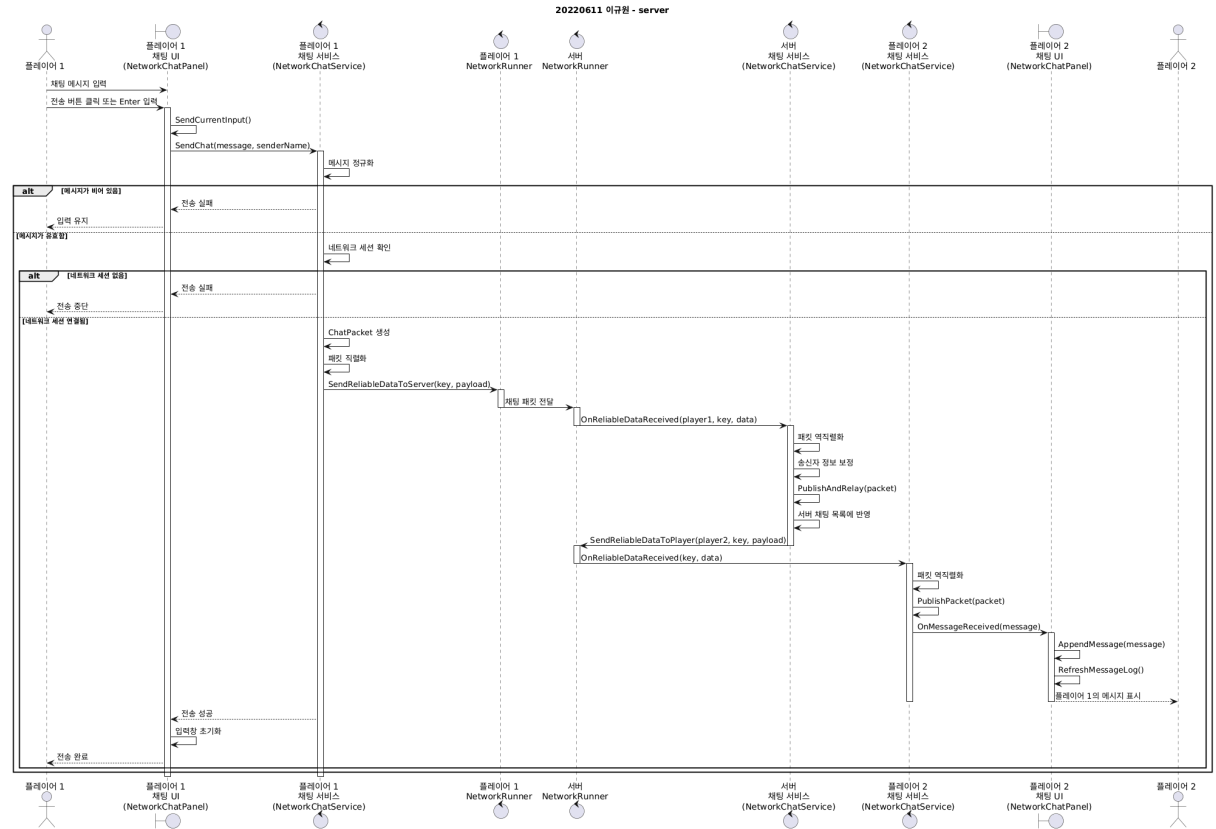
Server

UseCase 1: 방 입장하기

20220611 이규원 - server

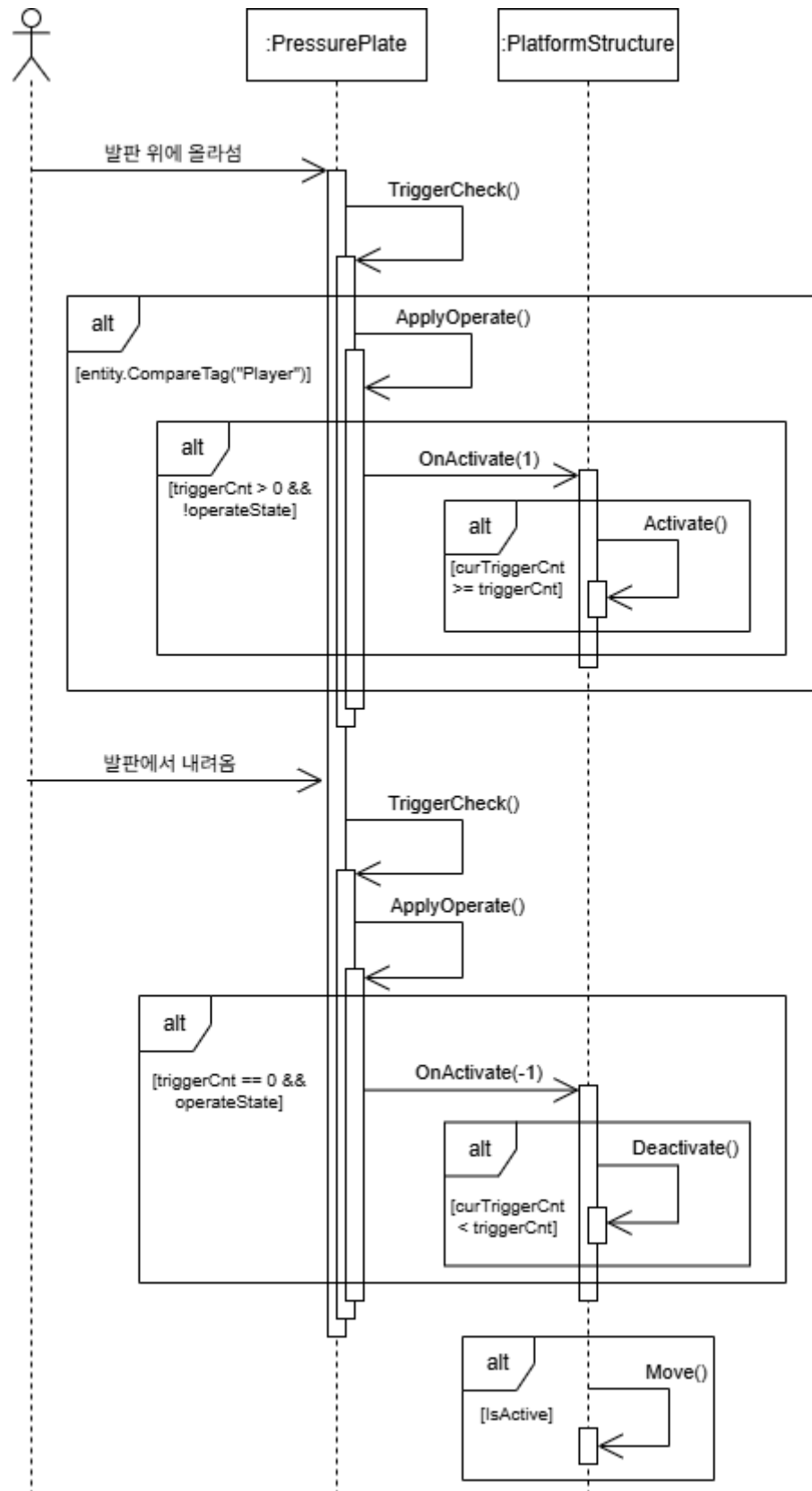


UseCase 2: 채팅 하기

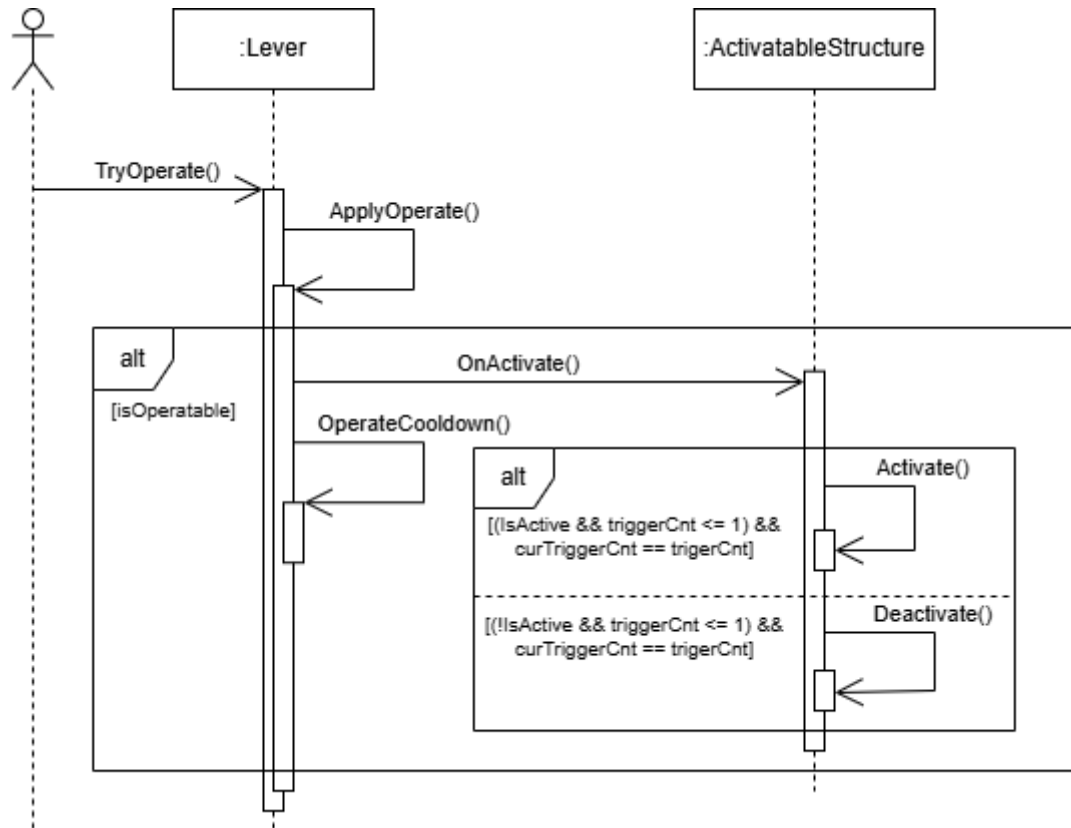


Object

UseCase 1: 레버 작동하기

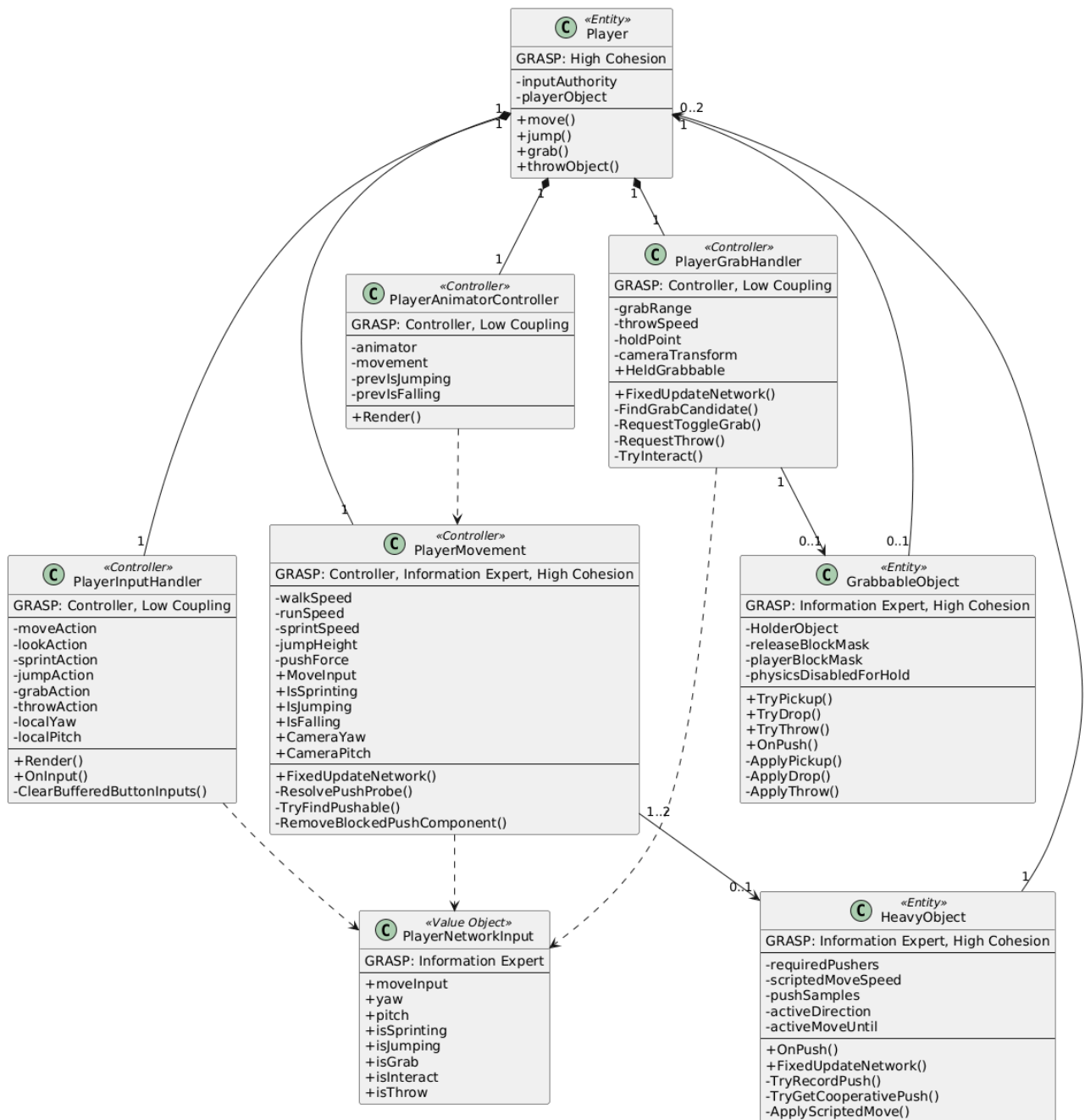


UseCase 2: 발판 작동하기



7-3. Class Diagram Player

Player Class Diagram



Conceptual class diagram과 차이점

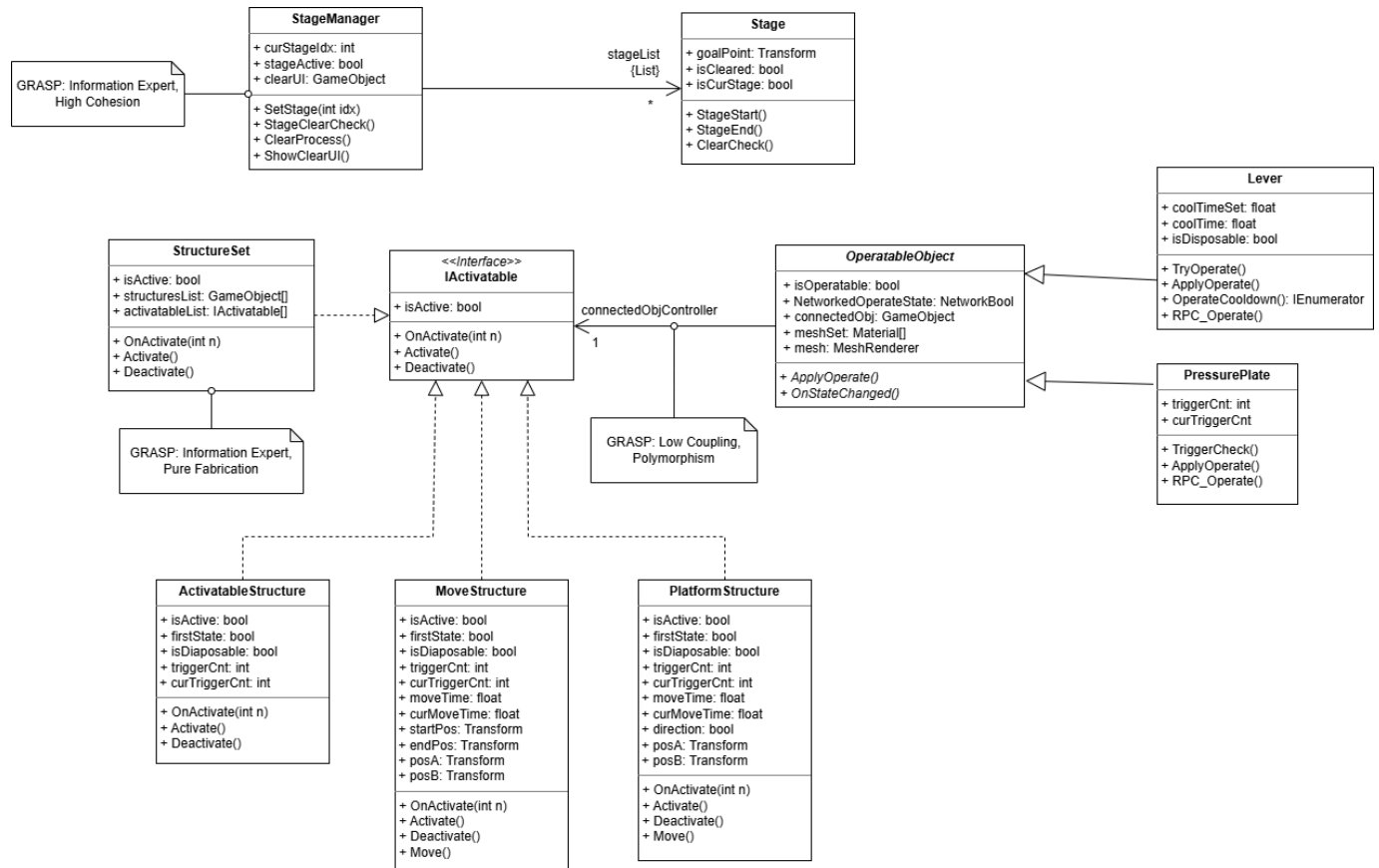
- PlayerInput이 PlayerInputHandler와 PlayerNetworkInput으로 나뉨
- MovementController를 PlayerMovement로 구체화하면서 FixedUpdateNetwork(), ResolvePushProbe() 같은 실제 네트워크 업데이트 메서드를 추가함
- PlayerNetworkInput이 추가되어 네트워크 입력 값을 별도 Value Object로 분리

20220611 이규원 - server



- 개념적 클래스 다이어그램은 RoomManager, Room, Player, GameSession 같은 핵심 도메인 객체 중심으로 구성됨.
- 설계 클래스 다이어그램은 Unity와 Photon Fusion 구현 환경을 반영해 NetworkSessionManager, LobbyManager, SceneFlowManager, NetworkRunner, SessionInfo 중심으로 변경됨.
- 개념 설계에서는 방 목록, 참가자 목록, 준비 상태, 게임 세션 상태를 직접 만든 클래스들이 소유하고 관리함.
- 구현 설계에서는 Room이나 GameSession 같은 순수 도메인 객체를 직접 유지하지 않고 Photon의 세션 정보와 NetworkRunner 상태로 대체함.
- 사용자의 방 생성 및 참가 요청은 LobbyManager가 받고, 실제 네트워크 처리는 NetworkSessionManager에 위임함.
- 개념 설계의 RuleManager는 준비 상태와 게임 시작 가능 여부를 판단하는 단일 책임을 가짐.
- 구현 설계에서는 이 책임이 WaitingRoomManager, RoomPlayerState, NetworkGameState 등 네트워크 동기화 가능한 클래스로 분산됨.
- 이러한 구조 변화는 멀티플레이 환경에서 각 플레이어의 상태를 모든 클라이언트에 동기화해야 하기 때문임.
- 결과적으로 설계 클래스 다이어그램은 개념 모델보다 기술 의존성이 강함.
- 대신 실제 네트워크 연결, 방 생성과 참가, 씬 전환, 상태 동기화를 수행할 수 있도록 책임과 의존 관계가 더 구체화됨.

Object



Conceptual class diagram과 차이점

- OperatableObject는 IActivatable 객체를 attribute로 가지므로 해당 인터페이스를 구현한 클래스가 아닌 인터페이스 자체와 관계를 연결하여 가시성 향상
- Button은 Lever와 가동 방식이 유사하고, 본래 기획했던 다회 상호작용 기능은 레버로도 구현 가능하다 판단, 그 역할을 레버가 흡수하고 Button 클래스는 제거
- OperatableObject와 그 관련 클래스들에 네트워크 상호작용을 위한 RPC 등 관련 변수, 메서드 추가
- OperatableObject가 여러 개의 IActivatable 객체를 제어해야 할 때, 해당 객체들을 배열로 관리하는 번거로움을 줄이고 단일 객체를 다루는 것처럼 할 수 있게 StructureSet 클래스를 추가